Estudiantes: Wilmary Soribel Hernández (10160486) y Asdruval de Jesús Tejada Pérez (10162340)

Informe de Análisis Asintótico y Complejidad Algorítmica

En este documento analizamos el rendimiento y la eficiencia de nuestro Sistema de Rutas, evaluando tanto la estructura de datos que construimos como los motores de búsqueda. Para medir esto formalmente, utilizamos las notaciones de cota superior (O), cota ajustada (Θ) y cota inferior (Ω), basándonos en la cantidad de paradas ($|V|$) y de rutas ($|E|$) de la red.

1. Operaciones sobre la estructura del Grafo

Para modelar la red de transporte, decidimos no usar la clásica matriz, sino un enfoque de Listas de Adyacencia apoyado en diccionarios de Java (HashMap y HashSet). Esto fue clave para lograr tiempos de respuesta casi instantáneos:

- **Inserción de una parada:**
  - **Complejidad:** Cota ajustada promedio $\Theta(1)$. Cota superior $O(|V|)$ (solo en caso extremo de colisión de hash).
  - **Justificación:** Insertar una nueva parada o ruta tiene un costo ajustado promedio de $\Theta(1)$. Como el HashMap ubica el ID directamente, nos ahorramos tener que iterar. En el peor de los casos (si hay una colisión de hash), la inserción de una parada podría caer a $O(|V|)$, pero es poco probable. Además, usar un HashSet nos solucionó el problema de las rutas duplicadas automáticamente.

- **Inserción de una ruta:**
  - **Complejidad:** Cota ajustada promedio $\Theta(1)$.
  - **Justificación:** Se accede al nodo de origen en tiempo constante $\Theta(1)$ y se inserta el objeto Ruta en su HashSet correspondiente. La estructura de conjunto previene rutas duplicadas automáticamente con costo amortizado constante.

- **Eliminación de una ruta:**
  - **Complejidad:** Cota superior $O(\text{grado}(v))$, donde $grado(v)$es el número de rutas que salen del origen.
  - **Justificación:** El borrado requiere ubicar el conjunto de conexiones del origen (lo cual es instantáneo en $\Theta(1)$ y luego utilizar el método removeIf. Este método debe iterar

secuencialmente evaluando cada arista vecina de ese vértice específico hasta encontrar y eliminar la conexión que apunta al destino deseado.

- **Consulta de vecinos de una parada:**
  - **Complejidad:** Cota inferior y ajustada $\Omega(1)$ / $\Theta(1)$.
  - **Justificación:** Esta es la operación más rápida del sistema. Acceder a la lista de adyacencia de cualquier nodo es directo, ya que el diccionario principal nos entrega el puntero a las colecciones vecinas en tiempo instantáneo sin importar qué tan grande sea el mapa.

- **Verificación de existencia de una arista:**
  - **Complejidad:** Cota superior $O\big(\text{grado}(v)\big)$.
  - **Justificación:** Al igual que en la eliminación, debemos recuperar la lista de adyacencia del nodo origen y realizar una búsqueda lineal entre sus conexiones activas para comprobar si existe alguna que apunte al destino que estamos evaluando.

2. Algoritmos Implementados

A continuación, se detalla el análisis de los algoritmos de recorrido y optimización, incluyendo los algoritmos especiales diseñados para cumplir con las reglas de negocio (transbordos y bloqueos dinámicos).

- **Búsqueda en Anchura (BFS) / Auditoría de Conectividad**
  - **Complejidad Temporal:** $\Theta(|V| + |E|)$
  - **Justificación:** Empleamos este algoritmo estrictamente como una herramienta de validación predictiva para asegurar que no existan "islas" o paradas huérfanas en el sistema. Logra su cota ajustada porque encola cada vértice del grafo y revisa cada arista una única vez, llevando el registro eficientemente mediante un HashSet auxiliar de nodos visitados.

- **Algoritmo de Dijkstra (Criterio: Tiempo, Distancia, Transbordos)**
  - **Complejidad Temporal:** $O\big((|V| + |E|)\log|V|\big)$

- **Justificación:** Al fundamentar el algoritmo sobre una PriorityQueue (Min-Heap), nos aseguramos de que extraer el nodo más prometedor tome $O(\log|V|)$). En el peor de los casos, procesamos toda la red. Sin embargo, optimizamos su rendimiento añadiendo un enfoque fast-fail el algoritmo se detiene inmediatamente al tocar el destino final, brindando una cota inferior real y eficiente de $\Omega(|V|\log|V|)$ para rutas cortas.

Algoritmo de Bellman-Ford (Criterio: Costo)

- **Complejidad Temporal:** $\Theta(|V| \times |E|)$
- **Justificación:** Dijkstra fracasa ante pesos negativos. Dado que nuestro modelo de negocio incluye posibles descuentos o subsidios tarifarios, tuvimos que implementar Bellman-Ford como solución segura. Debido a su naturaleza conservadora, no admite salidas tempranas y se ve obligado a relajar todas las aristas ($|E|$) repetidamente durante $|V| - 1$ ciclos, resultando en este costo estricto.

Algoritmo Especial 1: Yen / K-Shortest Paths (Rutas Alternativas)

- **Complejidad Temporal:** $O(K \times |V| \times (|V| + |E|)\log|V|)$
- **Justificación:** Para ofrecerle K opciones alternativas al usuario, Yen aplica bloqueos controlados de nodos y aristas (generando rutas (*Spur Paths*). Cada iteración obliga al sistema a lanzar un nuevo cálculo Dijkstra modificado desde los distintos vértices $|V|$ de la ruta original. Si bien es el proceso más costoso computacionalmente, el uso del Min-Heap interno nos permite recuperar la mejor alternativa calculada en $O(1)$.

Algoritmo Especial 2: APSP (All-Pairs Shortest Path propio)

- **Complejidad Temporal:** $O(|V| \times (|V| + |E|)\log|V|)$
- **Justificación:** Aunque la literatura recomienda Floyd-Warshall $O(|V|^3)$ para este escenario, decidimos descartarlo porque nos impedía rastrear multas acumulativas por transbordos. Nuestra solución fue orquestar una ejecución múltiple de Dijkstra desde cada vértice de la red. Considerando que los grafos de transporte urbano son dispersos (pocas aristas por nodo), esta arquitectura propia resulta ser más flexible ante nuestras reglas de negocio y marcadamente más veloz que la rigidez cúbica de Floyd-Warshall.

Nota: para acceder a la interfaz de administrador sin necesidad de código de verificación usar:

Usuario: admin
Contraseña: 1234